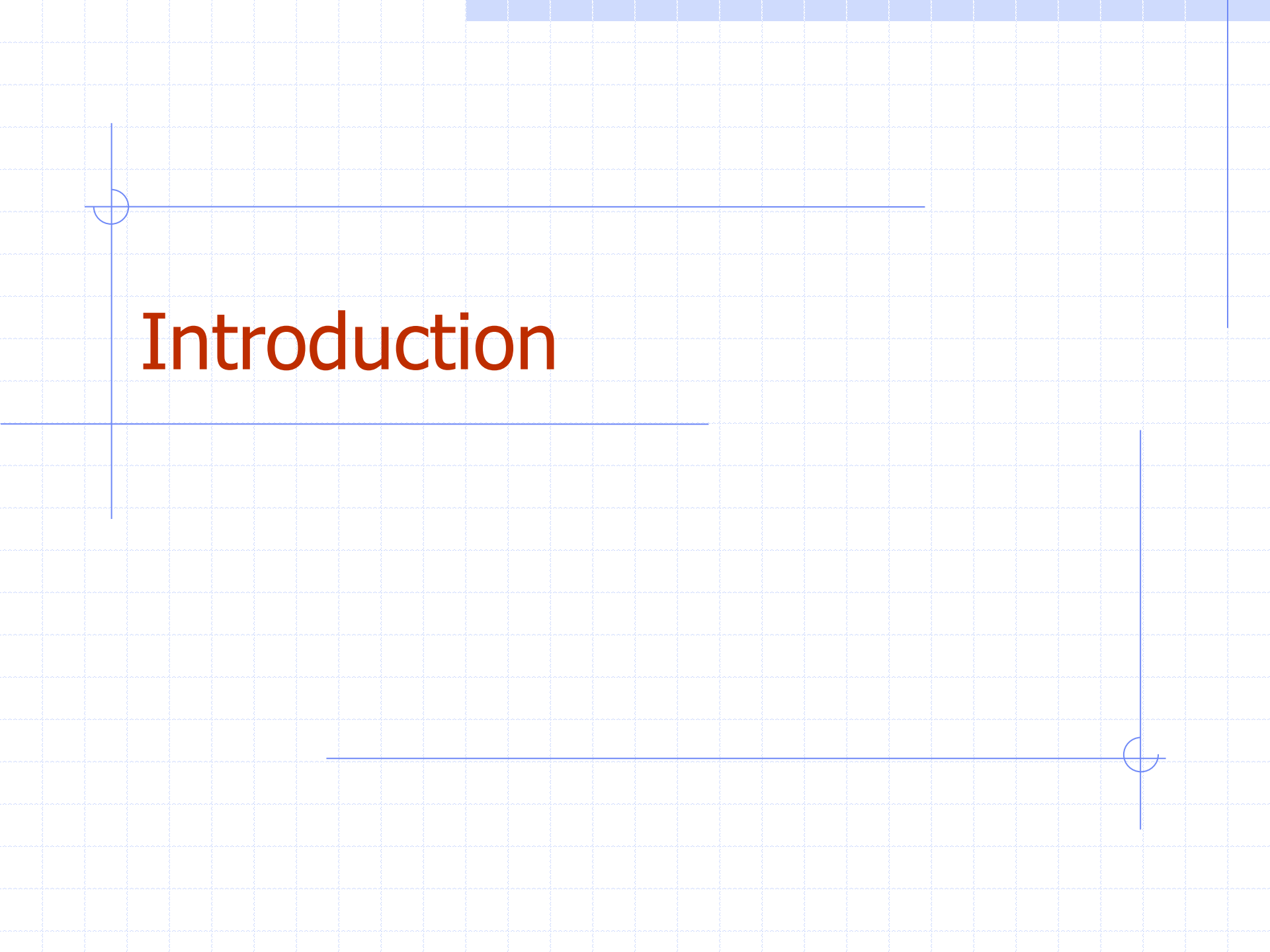
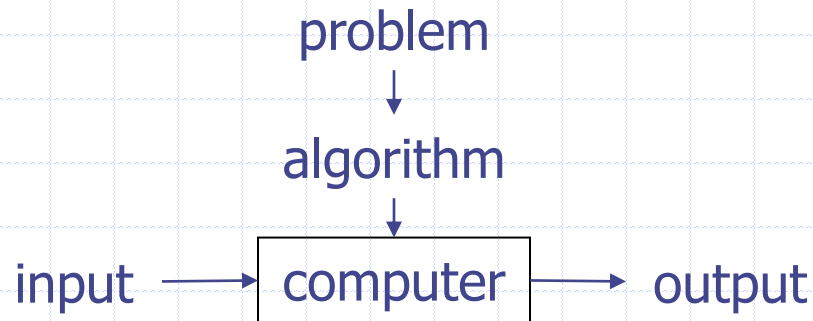


The background is a light gray grid. There are several blue lines: a vertical line on the left, a horizontal line across the top, a horizontal line across the middle, and a vertical line on the right. At the top-left intersection, there is a small blue circle with a horizontal line segment extending to the right. At the bottom-right intersection, there is a small blue circle with a horizontal line segment extending to the left.

Introduction

Why do we need to study algorithm

- ◆ We have to know a standard set of important algorithms from different areas of computing.
- ◆ We should be able to design a new algorithms and analyze their efficiency.
- ◆ The study of algorithms is recognized as the cornerstone of computer science
- ◆ There are useful in developing analytical skills



Fundamentals of Algorithmic Problem Solving

- ◆ Understanding the Problem
- ◆ Ascertain the Capabilities of a Computational Device
- ◆ Choosing between Exact and Approximate Problem Solving
- ◆ Deciding on Appropriate Data Structure
- ◆ Algorithm Design Techniques
- ◆ Methods of Specifying an Algorithm
- ◆ Proving an Algorithm's Correctness
- ◆ Analyzing an Algorithm
- ◆ Coding an Algorithm

Fundamentals of Algorithmic Problem Solving (Alternative)

- ◆ Problem definition
- ◆ Development of a model
- ◆ Specification of an Algorithm
- ◆ Designing an Algorithm
- ◆ Checking the correctness of an Algorithm
- ◆ Analysis of an Algorithm
- ◆ Implementation of an Algorithm
- ◆ Program testing
- ◆ Documentation

Fundamentals of Algorithmic Problem Solving

◆ Understanding the Problem

- Read the problem's description carefully and ask questions if needed.
- Do a few small examples by hand, think about special cases.
- Specify exactly the range of instances (input) the algorithm needs to handle.

◆ Ascertaining the Capabilities of a Computational Device

- RAM (random-access machine) model: Its assumption is that instructions are executed one after another, one at a time.
- Algorithms designed to be executed on such machines are called ***sequential algorithms***

Fundamentals of Algorithmic Problem Solving

◆ Choosing between Exact and Approximate Problem Solving

- In the former case, an algorithm is called an exact algorithm.
- However, there are important problems that simply cannot be solved, such as extracting square roots. Moreover, available algorithm for solving a problem exactly can be slow.

◆ Deciding on Appropriate Data Structures

- Appropriate data structure makes a algorithm to be more efficiency

◆ Algorithm Design Techniques

- There are several general design techniques (in this course).
- An algorithm design technique (or “strategy”) is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

Fundamentals of Algorithmic Problem Solving

◆ Methods of Specifying an Algorithm

- Using a natural language
- Pseudocode
- Flowchart

◆ Proving an Algorithm's Correctness

- We have to prove that the algorithm yields a required result for every legitimate input in a finite amount of time.
- A common technique for proving correctness is to use mathematics induction.
- To show that an algorithm is incorrect, we need to show only one instance of its input for which the algorithm fail.
- If algorithm is incorrect, we need to redesign data structure, algorithm technique and so on.

Fundamentals of Algorithmic Problem Solving

◆ Analyzing an Algorithm

- Time Efficiency (How fast the algorithm runs)
- Space efficiency (How much memory is being used)
- Simplicity
- Generality (Generality of the problem the algorithm solve and the range of inputs algorithm accepts)

◆ Coding an Algorithm

Important Problem Types

- ◆ There are problems in computing, but there are few areas that have attracted particular attention from researchers by the problem's practical importance or by some specific characteristics.
 - Sorting
 - Searching
 - String processing
 - Graph problems
 - Combinatorial problems
 - Geometric problems
 - Numerical problems

Important Problem Types

◆ Sorting

- Sorting makes many questions about the list easier to answer.
- There are many sorting algorithm, however there is no algorithm that would be the best solution in all situations.
- Two properties of sorting algorithms deserve special mention
 - ◆ Stable (Same GPA of two students)
 - ◆ In place (Do not require extra memory)

◆ Searching

- Finding a given value (called a search key) in the given set.
- There is no single algorithm that fits all situations best (Some are very fast but applicable only to sorted array).
- Linear Search, Binary Search, Interpolation Search, etc
- Data structures and algorithms should be chosen to balance the requirement.

Important Problem Types

◆ String Processing

- A string is a sequence of characters from an alphabet.
- One particular problem – that of searching for a given word in a text (String matching)
- Sets of strings:
 - ◆ Search trees, string sorting, binary search
- Exact string matching:
 - ◆ Finding a pattern (string) in a text (string)
- Approximate string matching:
 - ◆ Finding in the text something that is similar to the pattern

Important Problem Types

◆ Graph Problems

- A collection of points called vertices, some of which are connected by line segments called edges.
- Graphs can be used for modeling a wide variety of real-life applications (transportation and communication networks)
- Graph algorithms include graph traversal algorithms, TSP, graph coloring problem.

Important Problem Types

◆ Combinatorial Problems

- Problems that ask to find a combinatorial object – such as a permutation, a combination, or a subset.
- These are the most difficult problem in computing from both the theoretical and the practical.

◆ Geometric Problems

- Geometric problems deal with geometric objects such as points, lines, and polygons.
- Problems include
 - ◆ The closest pair
 - ◆ Convex hull problem.
 - ◆ How to check if two given line segments intersect?
 - ◆ Given n line segments, find if any two segments intersect
 - ◆ Find minimum radius such that atleast k point lie inside the circle

Important Problem Types

◆ Numerical Problems

- Problems involve mathematical nature: solving equations, computing definite integrals, evaluating functions.
- Problems in numerical computing, including linear algebra, numerical integration, statistics, and differential equations.
- Most mathematical problems are be solved only approximately.